Edge-AI Based Smart Access Control System: Real-Time Face Recognition on STM32H7 using Renode Simulation

Metin Irkıçatal* and Anıl Arda†
*Computer Engineering Department, Hacettepe University, Ankara, Türkiye
†Graduate School of Science and Engineering, Hacettepe University, Ankara, Türkiye
E-mails: *metinirkicatal@gmail.com, †anilarda.ee@gmail.com

***Abstract*—This project designs and implements a Smart Access Control System that performs real-time face recognition entirely on a microcontroller, eliminating the need for cloud connectivity. The target platform is the STM32H743ZI (Cortex-M7, 480 MHz, 2 MB Flash, 1 MB RAM). The software stack is built on FreeRTOS V11.1.0, providing deterministic task scheduling across four pipeline stages: image capture, preprocessing, inference, and access-control output. Pre-cropped and resized face patches (96x96 grayscale) are injected into the MCU frame buffer via a Renode memory-mapped camera peripheral. For recognition, a pre-trained INT8-quantized MobileNetV1 (25% width multiplier) is deployed using TensorFlow Lite for Microcontrollers (TFLite Micro) as a feature extractor. Embedding vectors from the penultimate layer are compared against pre-computed reference embeddings using cosine similarity. Key performance targets are end-to-end latency below 500 ms, model size under 300 KB, and combined RAM usage under 1 MB.**

***Index Terms*—edge AI, face recognition, TinyML, FreeRTOS, STM32H743ZI, TFLite Micro, Renode, embedded systems.**

I. Introduction

Cloud-connected biometric security systems introduce three fundamental risks: network latency, dependence on constant internet connectivity, and exposure of sensitive biometric data during transmission. These limitations are especially problematic in high-reliability environments such as hospitals, research laboratories, and industrial facilities, where access control must remain operational regardless of network status.

This project addresses these challenges by deploying face recognition entirely at the edge, on a resource-constrained Cortex-M7 microcontroller. The inference pipeline runs under FreeRTOS using TensorFlow Lite for Microcontrollers [1], with no cloud dependency. The Renode simulation framework [2] is used to validate the system in a reproducible, hardware-in-the-loop environment. The AT&T Face Database [3] serves as the evaluation benchmark.

Our approach is motivated by three goals: (1) privacy by design — all biometric processing occurs locally on the MCU; (2) cost and energy accessibility — an STM32H7-based system is orders of magnitude cheaper than an NVIDIA Jetson or PC-based solution; and (3) reliability — a fully self-contained embedded system provides access-control functionality even during complete network outages.

The remainder of this paper is organized as follows. Section II reviews related work. Section III defines the problem. Section IV describes the system architecture. Section V details the implementation methodology. Section VI outlines the experimental setup. Section VII discusses system-level considerations.

II. Related Work

We organize the related literature into four thematic groups: (1) TinyML benchmarking and model optimization for Cortex-M devices, (2) embedded face recognition systems, (3) simulation frameworks for embedded hardware validation, and (4) lightweight metric learning for identification.

TinyML benchmarking and model optimization. Banbury et al. [4] provide a comprehensive benchmark of ML inference on Cortex-M targets, demonstrating that INT8-quantized MobileNet variants achieve viable accuracy within tight memory and latency budgets. David et al. [1] introduce TensorFlow Lite for Microcontrollers, the inference runtime we adopt, and characterize its memory footprint across a range of Cortex-M devices. Warden and Situnayake [5] systematically survey deployment strategies — including post-training quantization and feature-extraction pipelines — for ultra-low-power MCUs. A common limitation of this group is the absence of RTOS integration and real-time scheduling analysis; the works treat inference as a standalone benchmark rather than as a pipeline stage with hard deadlines.

Embedded face recognition systems. Viola and Jones [6] established the foundations of real-time face detection using cascaded classifiers, but their approach targets CPU-class platforms and does not translate directly to Cortex-M constraints. Zanetti et al. [7] deploy a CNN-based face recognition pipeline on an embedded platform, reporting results on a Cortex-A processor (ARM Cortex-A53). Their work demonstrates the viability of CNN-based recognition at the edge but does not address the Cortex-M7 memory constraints ($\leq$1 MB SRAM), FreeRTOS scheduling, or hardware-in-the-loop simulation. Samaria and Harter [3] introduce the AT&T Face Database that we use as our evaluation benchmark, establishing a standard for compact face recognition evaluation.

Simulation frameworks for embedded validation. The Renode framework [2] supports cycle-accurate, deterministic simulation of embedded systems including Cortex-M

targets. Its composable platform model (`.repl` files) allows peripheral-level customization without requiring vendor-provided board support packages, which is critical for our project given version compatibility limitations with STM32H743 vendor files. To our knowledge, no prior published work uses Renode specifically for hardware-in-the-loop validation of an embedded ML inference pipeline.

Lightweight metric learning for identification. Rather than training a fixed-class classifier, metric learning approaches compute embeddings and compare them at inference time, enabling enrollment of new subjects without retraining. Warden and Situnayake [5] and Banbury et al. [4] both note that cosine similarity over pre-computed embeddings is a memory-efficient identification strategy for MCU deployment. Our work applies this approach to face identification on a Cortex-M7 target with FreeRTOS scheduling.

Positioning. As shown in Table I, our work is the only system in this comparison that combines Cortex-M7 deployment, FreeRTOS real-time scheduling, INT8 quantized inference, hardware-in-the-loop Renode simulation, and a metric learning (embedding + cosine similarity) recognition strategy. Prior embedded face recognition work targets higher-end platforms (Cortex-A), and prior TinyML benchmarking work does not integrate RTOS scheduling or simulation validation.

TABLE I
COMPARISON OF RELATED EMBEDDED FACE RECOGNITION AND TINYML WORKS

Work	Year	Cortex-M	RTOS	Simulation	INT8	Metric Learn.	Face Recog.
Viola & Jones [6]	2004	✗	✗	✗	✗	✗	✓
David et al. [1]	2019	✓	✗	✗	✓	✗	✗
Warden & Situnayake [5]	2019	✓	✗	✗	✓	✓	✗
Banbury et al. [4]	2021	✓	✗	✗	✓	✗	✗
Zanetti et al. [7]	2021	✗	✗	✗	✓	✗	✓
Antmicro [2]	2024	✓	✗	✓	✗	✗	✗
This Work	2026	✓	✓	✓	✓	✓	✓

III. PROBLEM DEFINITION

A. System / Application Description

The system performs face-based access control on a simulated STM32H743ZI microcontroller. A Renode inline Python peripheral injects pre-processed 96×96 grayscale face images from the AT&T Face Database into a memory-mapped camera address space. The MCU firmware reads these frames, runs an embedded ML inference pipeline under FreeRTOS, and outputs an accept or reject decision via GPIO and UART.

B. Objectives

- Deploy an INT8-quantized MobileNetV1 as a feature extractor on a Cortex-M7 MCU.
- Achieve end-to-end inference latency below 500 ms per frame.
- Fit the complete system (model + runtime + buffers) within 2 MB Flash and 1 MB SRAM.
- Validate recognition accuracy (FAR $< 5\%$, FRR $< 10\%$) on a held-out test set.
- Demonstrate deterministic, deadlock-free RTOS scheduling across 100 consecutive frames.

C. Constraints

- **Flash**: ≤ 2 MB (hardware limit of STM32H743ZI)
- **SRAM**: ≤ 1 MB (hardware limit of STM32H743ZI)
- **Latency**: < 500 ms end-to-end per frame
- **Model size**: ≤ 300 KB Flash (INT8 quantized)
- **No cloud connectivity**: all processing must occur on-device

IV. SYSTEM ARCHITECTURE AND DESIGN

As illustrated in Fig. 1, the system is organized into four sequential FreeRTOS tasks communicating through protected shared buffers. The Renode simulation environment hosts the camera peripheral and injects test frames into the MCU address space via DMA transfer.

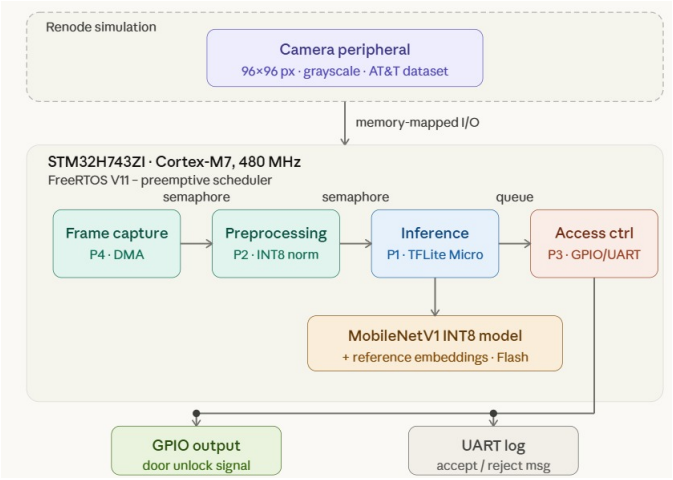

Fig. 1. High-level system architecture and data processing pipeline, illustrating the integration of the Renode simulation environment with the STM32H743ZI microcontroller and FreeRTOS task scheduling.

A. Hardware/Software Architecture

Hardware platform: STM32H743ZI (ARM Cortex-M7 @ 480 MHz, 2 MB Flash, 1 MB SRAM, DMA controller, USART3, GPIO).

Software stack: FreeRTOS V11.1.0 with preemptive scheduling; TFLite Micro runtime; Renode v1.16.0 simulation environment; minimal hand-crafted `.repl` platform description.

The four FreeRTOS pipeline tasks are:

1) **Task 1 — Frame Capture (Priority 4, highest):** DMA transfer moves a 96×96 grayscale frame from the memory-mapped Renode camera peripheral into a double-buffered SRAM region. Signals Task 2 via a binary semaphore on transfer complete.
2) **Task 2 — Preprocessing (Priority 2):** Normalizes pixel values to the INT8 range expected by the TFLite Micro input tensor. Signals Task 3 via semaphore once the buffer is formatted.
3) **Task 3 — TFLite Inference (Priority 1, lowest):** Loads the pre-quantized MobileNetV1 model from Flash, runs

inference, extracts the embedding vector, and computes cosine similarity against reference embeddings. Assigned the lowest priority because it is a long-running, CPU-intensive task, allowing capture and output tasks to preempt if necessary. Outputs subject label and confidence score via FreeRTOS queue.

4) **Task 4 — Access Control Output (Priority 3):** Applies a confidence threshold. On accept, drives a GPIO pin HIGH (simulated door unlock) and sends a UART log message. On reject, maintains GPIO LOW and logs the rejection.

B. DMA-Based Frame Buffer Management

The STM32H743ZI DMA controller is configured for memory-to-memory transfers, moving each incoming frame from the peripheral address space to the inference input buffer without CPU involvement. Double buffering ensures Task 1 can begin the next DMA transfer while Task 2 processes the previous frame, preventing pipeline stalls.

C. Renode Simulation Strategy

Due to compatibility limitations between Renode v1.16.0 and vendor-provided STM32H743 platform files (unsupported peripheral types such as `STM32_DMAMUX` and array-valued properties in newer `.repl` syntax), we adopt a minimal hand-crafted platform description containing only the peripherals required by the project: CPU, Flash, DTCMRAM, RAM, NVIC, and USART3. The camera peripheral is implemented as a memory-mapped Renode `PythonPeripheral` using an inline script, injecting frames from the AT&T Face Dataset into the peripheral address space.

D. Dataflow and System Model

Data flows unidirectionally through the pipeline: Renode camera peripheral $\rightarrow$ DMA $\rightarrow$ double frame buffer (SRAM) $\rightarrow$ preprocessing buffer $\rightarrow$ TFLite tensor arena $\rightarrow$ embedding comparison $\rightarrow$ access decision (GPIO/UART). Synchronization is enforced by FreeRTOS binary semaphores between Tasks 1–3 and a message queue between Tasks 3–4.

E. Requirements Analysis

1) Functional Requirements:

1) The system shall accept a 96×96 grayscale image frame from the simulated frame buffer.
2) The system shall run TFLite Micro inference and produce a recognition label within the latency budget.
3) The system shall output an accept or reject decision via GPIO and UART.
4) The system shall operate continuously under FreeRTOS task scheduling without deadlock or priority inversion.

2) Non-Functional Requirements:

1) End-to-end inference latency: < 500 ms per frame.
2) Total Flash usage (model + code): ≤ 2 MB.
3) Total SRAM usage (frame buffer + heap + stacks): ≤ 1 MB.
4) Task deadline (capture to output): < 1 s.
5) Recognition accuracy: FAR $< 5\%$, FRR $< 10\%$.

F. Timing and Execution Model

The pipeline is event-driven. Task 1 is triggered by a DMA transfer-complete interrupt. Subsequent tasks are triggered via FreeRTOS semaphores in a chain, forming a single-shot pipeline per frame. End-to-end latency is measured from DMA transfer complete (Task 1 start) to GPIO toggle (Task 4 completion) using the Cortex-M7 DWT cycle counter.

G. Model of Computation

We adopt a **Kahn Process Network (KPN) / dataflow model of computation** for this system. Each FreeRTOS task maps to a dataflow process node that consumes tokens (frames or tensors) from input channels and produces tokens into output channels. Communication channels are implemented as FreeRTOS semaphores (binary blocking) and message queues, which enforce the FIFO ordering and blocking-read semantics that characterize a KPN.

Why dataflow over alternatives:

Synchronous Reactive (SR) was considered but rejected: SR requires all processes to execute in a globally synchronized clock tick. Our system has variable-duration stages — inference (Task 3) takes orders of magnitude longer than frame capture (Task 1) — making a single global tick either wasteful (tick too long) or infeasible (tick too short for inference). SR is appropriate for control-dominated systems with short, uniform reaction times, not for a pipeline with a long-latency ML stage.

Event-driven (interrupt-only) model was also considered. While DMA completion does generate a hardware interrupt that triggers Task 1, an event-driven model alone does not naturally express the pipeline's producer-consumer ordering or the blocking behavior between Tasks 2, 3, and 4. Event-driven architectures can lead to callback hell and make deadlock analysis harder in multi-stage pipelines.

Hybrid justification: In practice, the system is a **hybrid**: the entry point (DMA interrupt $\rightarrow$ Task 1) is event-driven, while Tasks 1 through 4 form a dataflow chain. This is consistent with Ptolemy II's hybrid MoC composition, where an event-driven wrapper initiates a synchronous dataflow subgraph. The dataflow model is the dominant computational structure, governing the pipeline's determinism and deadlock properties.

H. Assumptions and Design Limitations

- Face detection is performed offline; only pre-cropped, pre-resized 96×96 patches are injected.
- The MobileNetV1 model is used as a fixed feature extractor without fine-tuning on the target dataset.
- Renode simulation operates cycle-accurately for the CPU but timing of the Python peripheral is approximate.
- The evaluation is performed entirely in simulation; physical hardware deployment is out of scope.

I. Resource Model

- TFLite Micro model (INT8 MobileNetV1 0.25): ∼150–250 KB Flash
- TFLite Micro runtime + tensor arena: ∼100 KB SRAM

- Double frame buffer: ∼18.4 KB SRAM ($96 \times 96 \times 1 \times 2$ bytes)
- Reference embeddings (40 subjects): ∼20 KB Flash
- FreeRTOS kernel + task stacks: ∼30 KB SRAM
- Remaining code and constants: ∼200–300 KB Flash

Total estimated usage stays within the 2 MB Flash / 1 MB SRAM limits, leaving headroom for iteration.

V. Methodology

A. Implementation Details

Embedded ML model: We use a pre-trained MobileNetV1 with width multiplier 0.25, quantized to INT8 using TensorFlow post-training quantization. The model is converted to a C byte array and compiled directly into Flash. Rather than performing direct N-class classification (which would require retraining on the AT&T dataset), the model serves as a feature extractor. The embedding vector from the penultimate layer is computed for each incoming frame and compared against pre-stored reference embeddings using cosine similarity. The subject with the highest similarity score above a tunable threshold is identified.

Reference embeddings for each of the 40 AT&T subjects are computed offline using 8 of the 10 images per subject and stored as a fixed array in Flash. The remaining 2 images per subject form the held-out test set for FAR/FRR evaluation.

Dataset: The AT&T Face Database [3] contains 400 grayscale images of 40 subjects (10 per subject) at 92×112 pixels. Images are resized to 96×96 using bilinear interpolation and saved as raw binary files on the host for injection via the Renode camera peripheral.

RTOS task design: All four pipeline tasks are assigned fixed priorities under FreeRTOS preemptive scheduling. Task 3 (inference) is assigned the lowest priority to prevent it from starving time-critical capture and output tasks. Inter-task communication uses FreeRTOS semaphores and message queues with no shared mutable state outside protected critical sections.

B. Algorithm Description

Algorithm summarizes the recognition pipeline executed per frame.

Input: Raw frame buffer F (96×96 grayscale, uint8)
Output: Identity label $\hat{y}$, decision $d \in \{\text{ACCEPT}, \text{REJECT}\}$
$F_{\text{norm}} \leftarrow \text{Normalize}(F)$ {INT8 preprocessing}
$\mathbf{e} \leftarrow \text{TFLiteMicro.Infer}(F_{\text{norm}})$ {Extract embedding}
for each subject $i \in \{1, \ldots, 40\}$ **do**
  $s_i \leftarrow \cos(\mathbf{e}, \mathbf{r}_i)$ {Cosine similarity}
end for
$\hat{y} \leftarrow \arg\max_i s_i$
if $s_{\hat{y}} \geq \tau$ **then**
  $d \leftarrow$ ACCEPT; drive GPIO HIGH; log via UART
else
  $d \leftarrow$ REJECT; maintain GPIO LOW; log via UART
end if

Algorithm . Per-frame face recognition pipeline.

C. Verification Strategy

Each pipeline stage is verified independently before integration:

- Frame injection verified by reading back the buffer via UART.
- Preprocessing verified by comparing output tensors against a reference Python implementation.
- Inference verified by running the same model in TFLite Python and comparing embedding outputs.
- End-to-end latency measured using the Cortex-M7 DWT cycle counter with a GPIO toggle at pipeline entry and exit.

VI. Experimental Setup

A. Platform and Tools

- **Simulator:** Renode v1.16.0 with hand-crafted STM32H743ZI `.repl`
- **RTOS:** FreeRTOS V11.1.0
- **ML runtime:** TensorFlow Lite for Microcontrollers (TFLite Micro)
- **Toolchain:** ARM GCC (arm-none-eabi), GNU Make
- **Host OS:** Ubuntu 22.04 (WSL2 on Windows)
- **Python:** v3.10 for offline preprocessing and embedding generation

B. Datasets / Workloads

The AT&T Face Database [3] (400 images, 40 subjects, 10 per subject) is used. Images are resized offline to 96×96 grayscale and saved as raw binary files. 8 images per subject are used for reference embedding generation; 2 images per subject (80 total) form the held-out test set injected via Renode.

C. Baselines

Three baselines are defined for comparison:

1) **Python TFLite baseline (host CPU):** The identical MobileNetV1 INT8 model and cosine similarity pipeline is executed in Python using the TFLite interpreter on the host CPU (Intel Core i5, x86-64). This establishes the accuracy upper bound for our recognition approach (since the model and embeddings are identical) and provides a latency reference point unaffected by MCU memory constraints.
2) **Float32 (unquantized) model:** A float32 version of MobileNetV1 0.25 is evaluated on the host to quantify the accuracy cost of INT8 post-training quantization. This isolates quantization degradation from other system factors.
3) **Threshold sensitivity sweep:** The cosine similarity threshold τ is varied over $[0.5, 0.95]$ in steps of 0.05 to characterize the FAR/FRR trade-off curve. This serves as a self-baseline for the access-control security tuning analysis.

The STM32H743ZI embedded implementation is expected to match the Python TFLite baseline in recognition accuracy

(since both use the same INT8 model and embeddings) while demonstrating that the 500 ms latency target is achievable on the constrained hardware.

D. Evaluation Metrics

- **Inference Latency:** Time (ms) from DMA transfer complete to access decision output, measured via the Cortex-M7 DWT cycle counter. Reported as mean and worst-case over 80 test frames. Target: < 500 ms. Compared against Python TFLite baseline latency.
- **Memory Utilization:** Actual Flash and SRAM consumption measured via linker map (static) and runtime heap profiling (dynamic). Target: within budget defined in Section IV-I.
- **Recognition Accuracy (FAR / FRR):** Evaluated on the held-out 2-image-per-subject test set (80 images, 40 subjects). Target: FAR $< 5\%$, FRR $< 10\%$. Compared against Python TFLite baseline and float32 baseline.
- **Scheduling Correctness:** No task deadline miss across 100 consecutive frame injections in Renode simulation. Reported as deadline miss rate.

VII. System-Level Considerations

Real-time constraints and latency: The 500 ms end-to-end latency target is enforced by the FreeRTOS task chain. Task 3 (inference) is the critical-path bottleneck; its priority is deliberately set lowest to allow preemption by the capture and output tasks if needed.

Reliability and fault tolerance: The double-buffered frame buffer prevents pipeline stalls if a DMA transfer completes while preprocessing is ongoing. Deadlock is prevented by unidirectional semaphore signaling with no circular dependencies. Priority inversion is mitigated by the chosen priority assignment.

Energy efficiency: The Cortex-M7 DMA-based frame transfer eliminates CPU cycles during data movement. Inference is performed on demand (event-driven), not continuously polled. Future work could explore MCU sleep modes between frames.

Security considerations: All biometric processing occurs locally; no data is transmitted over any network. The cosine similarity threshold τ is a tunable security parameter — raising τ decreases FAR at the cost of higher FRR.

Scalability: The current design supports 40 enrolled subjects. Scaling to more subjects requires additional Flash for reference embeddings ($\sim$0.5 KB per subject) but does not change the inference runtime or SRAM usage. Re-enrollment does not require model retraining.

Privacy: By design, no biometric data leaves the device, directly satisfying GDPR and privacy-by-design principles.

Acknowledgments

The authors thank the Embedded Systems Design course instructors at Hacettepe University for their guidance and feedback on this project proposal.

References

[1] R. David, J. Duke, A. Jain, V. J. Reddi, N. Jeffries, J. Li, N. Kreeger, I. Nappier, M. Natraj, T. Wang, P. Warden, and R. Rhodes, "TensorFlow Lite Micro: Embedded machine learning on TinyML systems," *arXiv preprint arXiv:1910.08602*, 2019.

[2] Antmicro, "Renode: An open source framework for complex embedded systems simulation," Official Documentation, 2024. [Online]. Available: https://renode.io

[3] F. S. Samaria and A. C. Harter, "Parameterisation of a stochastic model for human face identification," in *Proceedings of the 2nd IEEE Workshop on Applications of Computer Vision*, 1994.

[4] C. Banbury, V. J. Reddi, M. Lam, W. Fu, A. Fazel, J. Holleman, X. Huang, R. Hurtado, D. Kanter, A. Lokhmotov, D. Patterson, D. Pau, J. sun Seo, J. Sieracki, U. Thakker, M. Verhelst, and P. Yadav, "Benchmarking TinyML systems: Challenges and direction," *arXiv preprint arXiv:2003.04821*, 2021.

[5] P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media, 2019.

[6] P. Viola and M. Jones, "Robust real-time face detection," *International Journal of Computer Vision*, vol. 57, no. 2, pp. 137–154, 2004.

[7] M. Zanetti, A. Bartolini, and L. Benini, "Real-time face recognition on embedded platforms," in *Proceedings of the Design, Automation and Test in Europe Conference (DATE)*, 2021.